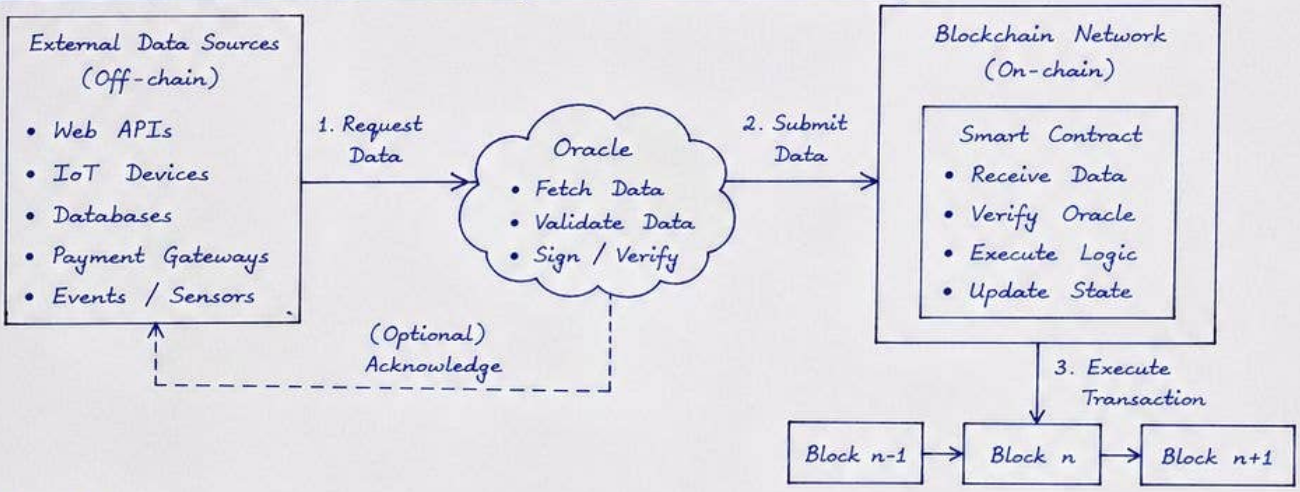
1. Explain a simplified model of an oracle interacting with smart contract on block chain.

15 Marks

Answer:

An oracle is an external service that provides real world data or off-chain computation to a smart contract on the blockchain. Since smart contracts cannot access external data by themselves, oracles act as a bridge between on-chain and off-chain world.

Simplified Model of Oracle interacting with Smart Contract:



Working Steps:

1) The smart contract sends a request to the oracle for particular external data.
2) The oracle collects the data from external sources, validates and signs it.
3) The oracle submits the data to the smart contract through a transaction.
4) The smart contract verifies the oracle (using its address or signature).
5) If valid, the smart contract executes the required logic and updates the blockchain state.
6) The transaction is recorded on the blockchain and becomes immutable.

Key Points:

- Oracles enable smart contracts to interact with real world data.
- They ensure data authenticity through validation and digital signatures.
- Multiple oracles can be used to avoid single point of failure.
- Common in DeFi, insurance, supply chain, gaming, IoT applications.

Advantages:

- Extends smart contract capability beyond blockchain.
- Enables automation based on real world events.
- Maintains decentralization when using multiple oracle nodes.

Limitations:

- Oracle can be a single point of failure if centralized.
- Data quality and correctness depend on oracle.
- Additional cost and delay for fetching data.

Conclusion:

Thus, the oracle acts as a trusted intermediary between external world and blockchain, enabling smart contracts to execute real world driven logic securely.

—— X ——

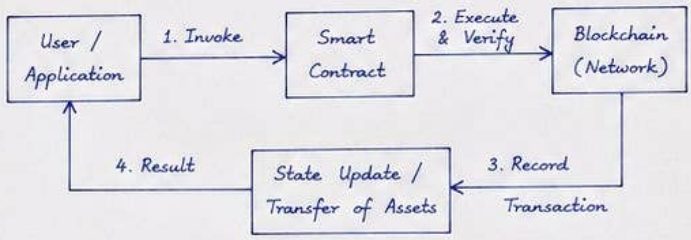
2. Briefly discuss about smart contracts on a blockchain.

15 Marks

Answer:

A smart contract is a self-executing program stored on a blockchain that automatically enforces and executes the terms of an agreement when predefined conditions are met. It eliminates the need for intermediaries and provides a trustless and transparent way to transact.

How it works:



Components:

- Code : Contains the business logic and rules.
- Data : Stores contract state and variables.
- Events : Emit logs for off-chain consumption.
- Functions : Define operations that can be invoked.

Characteristics / Features:

- Deterministic : Same input gives same output.
- Transparent : Code and transactions are visible.
- Immutable : Once deployed, cannot be changed.
- Trustless : No need for central intermediary.
- Autonomous : Executes automatically.

Advantages:

- Reduces cost by removing intermediaries.
- Increases speed and efficiency.
- Provides transparency and auditability.
- Minimizes fraud and errors.
- Available 24/7 and works globally.

Limitations:

- Bugs in code cannot be modified easily.
- Limited by underlying blockchain scalability.
- Legal enforceability of code is still evolving.
- Requires secure and correct oracles for external data.

Applications / Use cases:

- DeFi (lending, borrowing, swaps)
- Supply chain management
- Insurance claims
- Voting systems
- Digital identity and property records

Conclusion:

Smart contracts are a powerful application of blockchain technology that allows agreements to be executed in a decentralized, secure and efficient manner without relying on trusted third parties.

—— X ——

3. What is DAO? Discuss.

15 Marks

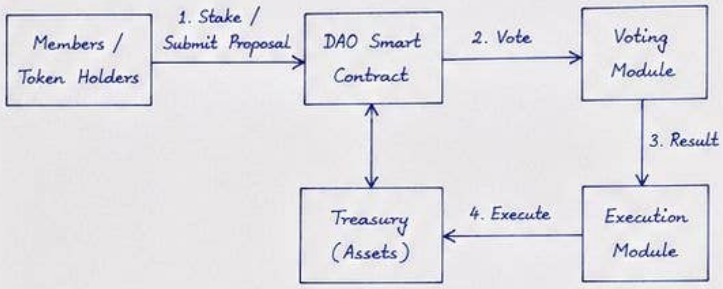
Answer:

A DAO (Decentralized Autonomous Organization) is an organization represented by rules encoded as a transparent computer program, controlled by the organization members and not influenced by a central government. It operates on a blockchain using smart contracts.

Key idea:

- No central authority.
- Decisions are made collectively by token holders.
- Funds are managed by smart contracts.
- Transparent, censorship resistant and immutable.

Architecture of DAO:



Working:

1) Members stake governance tokens and submit proposals.
2) Token holders vote on proposals (weight based on tokens).
3) If proposal gets majority, the result is passed to execution module.
4) Smart contract automatically executes the proposal and updates the treasury / state.

Advantages:

- Decentralized governance.
- Transparent and auditable.
- Community driven decision making.
- Immutable execution of rules.
- Global participation.

Challenges / Limitations:

- Smart contract bugs can be exploited.
- Low voter participation.
- Governance attacks (whale dominance).
- Legal and regulatory uncertainty.

Examples of DAO:

- The DAO (2016) - Early DAO on Ethereum.
- MakerDAO - Decentralized stablecoin governance.
- Uniswap DAO - Governs Uniswap protocol.
- Aave DAO - Governs lending protocol.

Conclusion:

DAO is a new way of organizing and managing resources on the blockchain using smart contracts. It enables decentralized governance and collective decision making without relying on intermediaries.

—— X ——

4. What is a wallet in Bitcoin? Explain in detail about the different types of wallet with example.

15 Marks

Answer:

A Bitcoin wallet is a software or hardware tool that stores the private keys used to access and manage Bitcoin. It does not actually store Bitcoins. Bitcoins are always stored on the blockchain. The wallet enables users to send, receive and view their balances.

Types of Bitcoin Wallets:

Type of Wallet	Description	Example
1. Software Wallet	• Runs on devices like phone or computer. • Stores private keys in digital form. • Convenient for daily transactions.	Ex: • Electrum • Exodus • Bitcoin Core Wallet
2. Hardware Wallet	• Physical devices that store private keys offline. • More secure as keys are not exposed to internet. • Used for large balances and long term storage.	Ex: • Ledger Nano S • Trezor • KeepKey
3. Web Wallet (Online Wallet)	• Wallets provided by third party services. • Accessed through web browsers. • Convenient but less secure as keys are held by service provider.	Ex: • Blockchain.com Wallet • Coinbase Wallet
4. Mobile Wallet	• Runs on smartphone apps. • Supports scanning QR codes and easy payments. • Suitable for everyday small transactions.	Ex: • Mycelium • Trust Wallet • BlueWallet
5. Paper Wallet	• Private and public keys are printed on paper. • Keys are offline and immune to hacking. • Must be kept safe from physical damage or loss.	Ex: • BitAddress.org • WalletGenerator.net
6. Cold Wallet	• Private keys stored offline. • Includes hardware wallets and paper wallets. • Highly secure.	Ex: • Ledger • Paper Wallet
7. Hot Wallet	• Private keys stored online. • More convenient but more vulnerable. • Used for frequent transactions.	Ex: • Electrum • Coinomi Wallet

Key Functions of a Wallet:

1. Generate and manage public and private keys.
2. Create Bitcoin addresses to send and receive payments.
3. Sign transactions using private keys.
4. Broadcast transactions to the Bitcoin network.
5. View balance and transaction history.

Conclusion:

Wallets are essential for interacting with Bitcoin. The choice of wallet depends on the trade-off between security and convenience.

— X —

5. Write a short note on: Privacy and Anonymity.

15 Marks

Answer:

Privacy and anonymity are fundamental concepts in blockchain systems. They ensure that users' identities and transaction details are protected from unwanted exposure while maintaining transparency and trust.

1. Privacy:

- Privacy refers to the control over personal information and the ability to restrict access to data.
- In blockchain, all transactions are recorded on a public ledger which is transparent.
- Privacy mechanisms are used to hide the identity of users and the details of transactions.
- Techniques for privacy in blockchain:
  a) Pseudonymous Addresses: Users are identified by addresses, not by real world identity.
  b) Encryption: Data is encrypted so that only authorized parties can view it.
  c) Zero Knowledge Proofs (ZKP): Prove a statement without revealing the underlying data.
  d) Confidential Transactions: Hide transaction amounts and details (e.g., in Monero, Zcash).

2. Anonymity:

- Anonymity means that the real identity of the user is unknown to others in the network.
- Bitcoin provides pseudonymity, not full anonymity.
- Anonymity is important to protect users from tracking, profiling and unwanted surveillance.
- Techniques to achieve anonymity:
  a) Mixing Services / Tumblers: Mix coins from many users to break the link between sender and receiver.
  b) CoinJoin Transactions: Combine multiple transactions into one to hide the flow of funds.
  c) Privacy Coins: Cryptocurrencies like Monero, Zcash provide strong anonymity by default.

3. Privacy vs Transparency:

Goal is to achieve a balance between transparency and privacy.

4. Importance:

- Protects user identity and financial information.
- Prevents unauthorized tracking and data misuse.
- Encourages adoption of blockchain technology.
- Helps in complying with data protection regulations.

Examples:

- Bitcoin – Pseudonymous (addresses are public).
- Monero – Strong privacy using ring signatures, stealth addresses.
- Zcash – Uses Zero Knowledge Proofs for private transactions.

Conclusion:

Privacy and anonymity are crucial in blockchain to protect users while keeping the system transparent, secure and trustworthy.

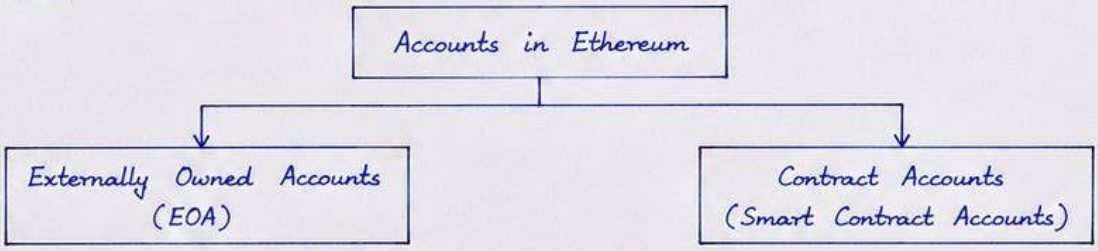
— X —

6. What are the two types of accounts that exists in Ethereum transaction.

15 Marks

Answer:

In Ethereum, all entities that can send transactions or hold balances exist as accounts on the state of the blockchain. There are two types of accounts.



1. Externally Owned Accounts (EOA):

- These accounts are owned and controlled by private keys.
- They are created by users and are not associated with any code.
- EOAs can send transactions, transfer Ether (ETH) and interact with smart contracts.
- They have nonce, balance and storage (storage is always empty for EOA).
- Example: A user account created using MetaMask or MyEtherWallet.
- Represented by address derived from a public key (last 20 bytes of Keccak-256 hash of the public key).

2. Contract Accounts (Smart Contract Accounts):

- These accounts are associated with smart contracts and exist on the blockchain.
- They are created when a smart contract is deployed.
- They do not have private keys; instead they are controlled by the code (contract) stored at that address.
- They can receive Ether, store data and execute code.
- They have nonce, balance, storage and code.
- Example: Deployed ERC-20 token contract, DAO contract.
- Address is derived from the sender's address and nonce (using RLP encoding and Keccak-256 hash).

Comparison:

Feature	Externally Owned Account (EOA)	Contract Account (Smart Contract)
1. Controlled by	Private Key of the user	Smart contract code.
2. Created by	User	Deployment transaction
3. Can initiate transaction.	Yes	No (can only respond)
4. Stores Code	No	Yes
5. Stores Data	No (storage empty)	Yes (storage available)
6. Can hold Ether	Yes	Yes
7. Example	User account in MetaMask	ERC-20 Token Contract

Conclusion:

Thus, Ethereum has two types of accounts: Externally Owned Accounts (EOA) which are controlled by users and Contract Accounts which are controlled by smart contract code. Both types of accounts are essential for Ethereum transactions and smart contract execution.

——— X ———

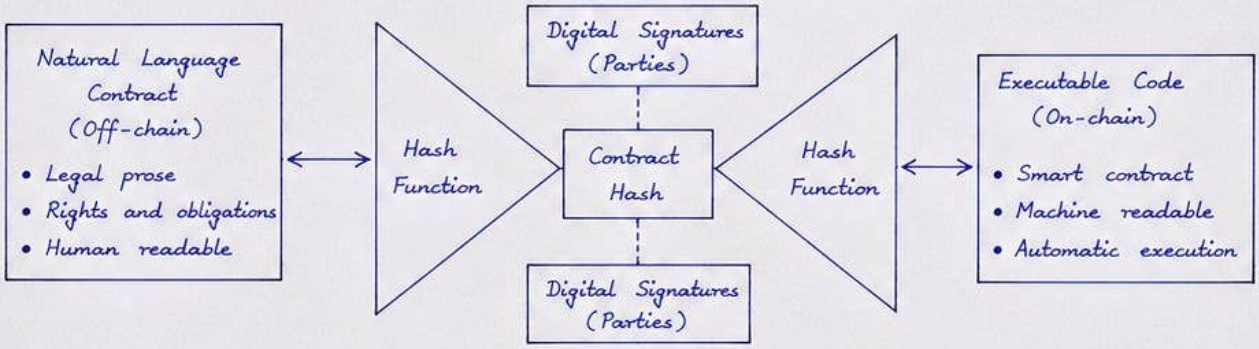
7. With a neat bowtie model diagram explain Ricardian Contracts.

15 Marks

Answer:

Ricardian Contracts are legal smart contracts. They link the legal terms of a contract written in natural language with the computer code that executes those terms. They use digital signatures to connect the parties, the legal document and the executable contract. The Bowtie model proposed by Ian Grigg is used to represent this.

Bowtie Model of Ricardian Contract



Working:

1) The parties first create a legal contract in natural language (off-chain).
2) The document is hashed using a cryptographic hash function to produce a unique hash.
3) The executable code (smart contract) that represents the obligations is also hashed.
4) Both hashes together with the identities of the parties are digitally signed.
5) The hashes and signatures are recorded on the blockchain.
6) The actual legal document may be stored off-chain (or on IPFS etc.).
7) When the smart contract is executed on-chain, anyone can verify that the code matches the legal contract using the recorded hashes and signatures.

Key Features:

- Links legal intent with executable code.
- Ensures integrity - any change in legal contract or code will change the hash.
- Provides non-repudiation through digital signatures.
- Enables automation while keeping the legal validity.

Advantages:

- Reduces ambiguity between legal agreement and code.
- Increases trust and compliance.
- Facilitates automatic execution of legally valid contracts.
- Suitable for enterprise and cross-border transactions.

Applications:

Supply chain agreements, insurance contracts, trade finance, property transfer, employment agreements, financial derivatives etc.

Conclusion:

Ricardian Contracts bridge the gap between legal contracts and smart contracts by using the bowtie model. They ensure that the legal agreement and the executable code are cryptographically linked, verifiable and tamper-proof, enabling trusted automation on blockchain.

X

8. Explain any five standard fields in Ethereum transaction.

15 Marks

Answer:

An Ethereum transaction is a piece of data that is signed by the sender's private key and sent to the Ethereum network. It contains several standard fields. Any five of them are explained below.

Sl. No.	Field	Description	Purpose / Role
1.	From (Sender Address)	20-byte Ethereum address of the account initiating the transaction.	• Identifies the sender. • Used to deduct balance and for transaction signature.
2.	To (Recipient Address)	20-byte address of the account that will receive the Ether or execute the contract.	• If it is a normal transfer, it is the receiver. • If it is a contract creation, this field is left empty.
3.	Value	Amount of Ether (in wei) to be transferred from sender to the recipient.	• Used for Ether transfer. • For contract calls, value can be zero.
4.	Gas Limit	Maximum amount of gas units the sender is willing to provide for the transaction.	• Sets the upper limit of computation work. • Prevents infinite loop and DoS attacks.
5.	Gas Price	Amount of Ether (in wei) the sender is willing to pay per gas unit.	• Determines the transaction fee (gas price x gas used). • Higher gas price gives higher priority to miners.

Other important standard fields (for reference):

- Nonce : Number of transactions sent by the sender account.
- Data : Input data for contract execution or additional information.
- Chain ID : Identifies the Ethereum network (mainnet, testnet etc.).
- R, S, V : Components of the sender's digital signature.

Summary:

These fields together define the intent, control the execution and ensure the security of an Ethereum transaction.

Conclusion:

The standard fields in Ethereum transactions are essential for transferring value, executing smart contracts and maintaining the security and efficiency of the network.

——— X ———

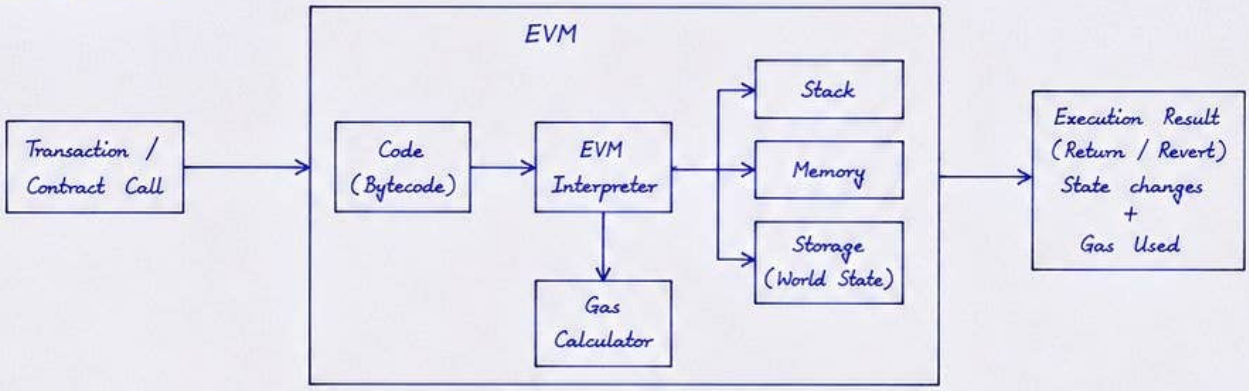
9. Briefly discuss about the operation of Ethereum Virtual Machine (EVM) with the help of a neat diagram.

15 Marks

Answer :

Ethereum Virtual Machine (EVM) is a runtime environment for smart contracts in Ethereum. It executes the bytecode of smart contracts in a distributed, deterministic and isolated manner on every Ethereum node.

Operation of EVM :



1. A transaction or a contract call is sent to the Ethereum network.
2. The EVM receives the transaction and fetches the bytecode of the target smart contract.
3. The EVM interpreter reads bytecode instructions one by one.
4. It uses Stack, Memory and Storage to perform operations.
   - Stack : temporary data (LIFO structure).
   - Memory : temporary read / write memory during execution.
   - Storage (World State) : persistent key - value database (blockchain state).
5. For every operation, Gas is consumed. Gas calculator keeps track of gas used.
6. If the execution completes successfully, results are returned and state changes are written to the blockchain. If any error occurs or gas is insufficient, execution is reverted.

Components of EVM :

- Code (Bytecode) : Compiled smart contract code (EVM bytecode).
- Interpreter : Executes bytecode instruction set.
- Stack : Holds 256-bit words used by the EVM (max 1024 items).
- Memory : Linear volatile memory, initialized to zero.
- Storage : Persistent storage accessible through account storage (Merkle Patricia Trie).
- Gas Calculator : Measures computational work and prevents abuse.

Features of EVM :

- Deterministic : Same input → same output on every node.
- Isolated : Runs in a sandbox; cannot access host system.
- Stateless between calls : Only storage persists between transactions.
- Secure : Gas mechanism, limits loops and malicious behavior.

Conclusion :

EVM provides a decentralized execution environment for smart contracts. It ensures that every node in the network executes the same code in the same way, maintaining consistency, security and trust in Ethereum.

— X —

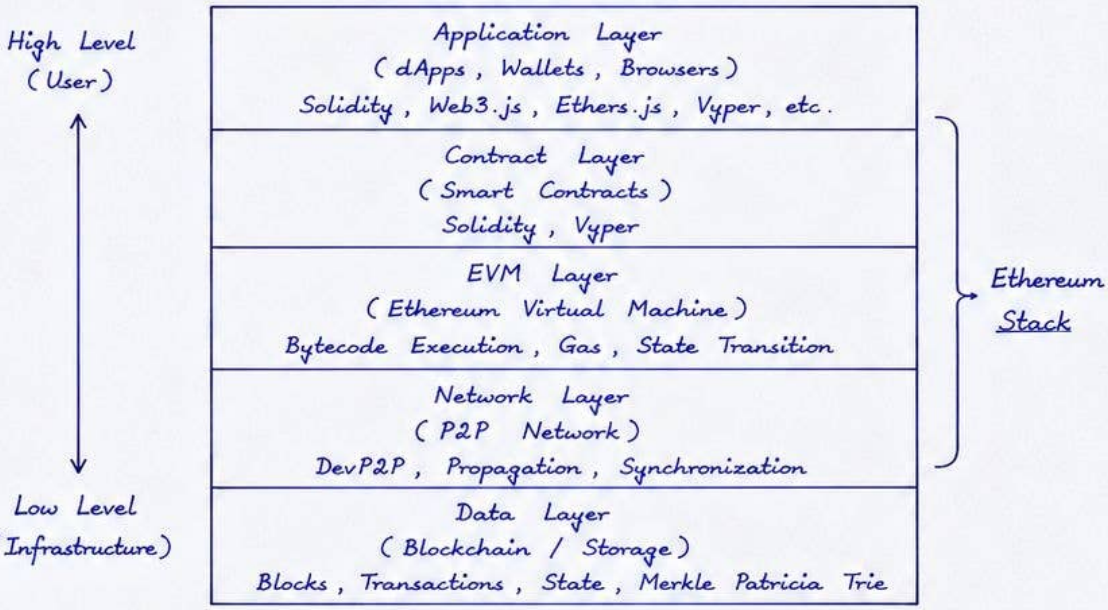
10. Discuss the Ethereum Stack with a suitable diagram.

15 Marks

Answer:

The Ethereum Stack is a layered architecture that shows how different components work together to enable decentralized applications (dApps) on Ethereum.

Ethereum Stack



Explanation of Layers:

1. **Application Layer:** Provides interfaces and tools for users to interact with Ethereum. Includes dApps, wallets (MetaMask), browsers and libraries (Web3.js, Ethers.js).
2. **Contract Layer:** Contains smart contracts written in high level languages like Solidity or Vyper. These are compiled into EVM bytecode.
3. **EVM Layer:** Ethereum Virtual Machine executes the bytecode in an isolated environment. It performs operations, manages gas and performs state transitions.
4. **Network Layer:** Handles communication between nodes. Uses DevP2P protocol for peer discovery, block propagation and synchronization.
5. **Data Layer:** Stores all blockchain data including blocks, transactions and world state. Uses Merkle Patricia Trie for efficient storage and verification.

Key Points:

- The stack shows how a transaction flows from the application down to the data layer and back.
- Each layer provides services to the layer above and uses services of the layer below.
- Together they enable secure, decentralized and deterministic execution of dApps.

Conclusion:

The Ethereum Stack provides a complete framework from user applications down to data storage and network communication, enabling decentralized computation and state management on Ethereum.

——— X ———

11. What is Ethereum Blockchain? Discuss with a state transition diagram. 15 Marks

Answer :

Ethereum Blockchain is a decentralized, distributed ledger that maintains the world state of all accounts, smart contracts, transactions and logs. It provides the underlying infrastructure for executing smart contracts in a trustless environment. It is a chain of blocks where each block contains a list of transactions and the resulting state root.

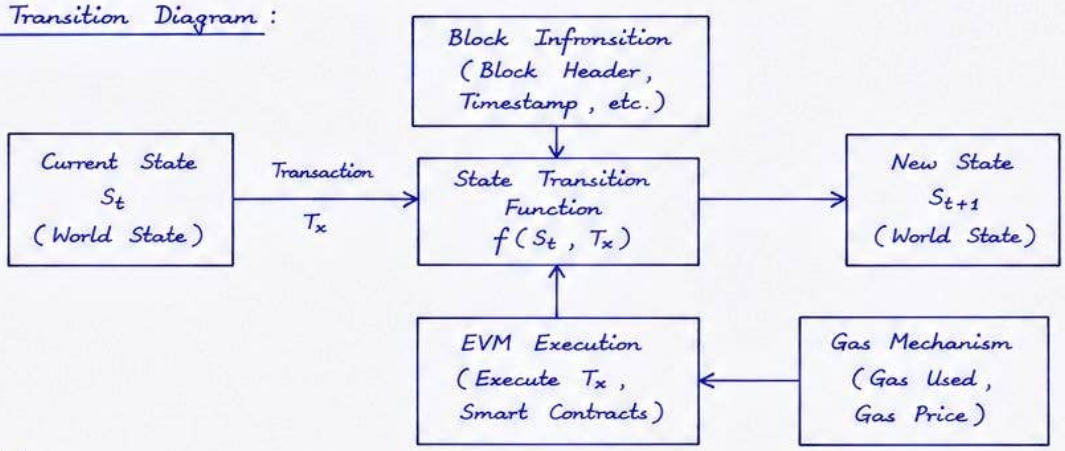
Key Features :

- It is public and permissionless.
- It maintains the world state as a mapping of accounts to their balances, nonce, code and storage.
- It uses Proof of Work (PoW) / Proof of Stake (PoS) for consensus.
- It enables execution of smart contracts through EVM.

State Transition in Ethereum :

Ethereum can be viewed as a state machine. The state of the Ethereum blockchain moves from one valid state to another by applying transactions according to the state transition function.

State Transition Diagram :



Explanation :

1. The current state S_t consists of all account balances, nonce, code and storage.
2. A transaction T_x is submitted to the network.
3. The state transition function f takes S_t, T_x and block information (timestamp, coinbase, etc.) as input.
4. The EVM executes the transaction. It runs the smart contract code and applies the rules. Gas is consumed for computation.
5. The gas mechanism determines the cost of computation (gas used × gas price).
6. If execution is successful, a new state S_{t+1} is produced and becomes the current state for the next transaction. Otherwise the transaction is reverted.

Conclusion :

Thus Ethereum Blockchain is a decentralized state machine where every valid transaction causes a state transition from S_t to S_{t+1} by executing the transaction through the EVM and following the gas mechanism.

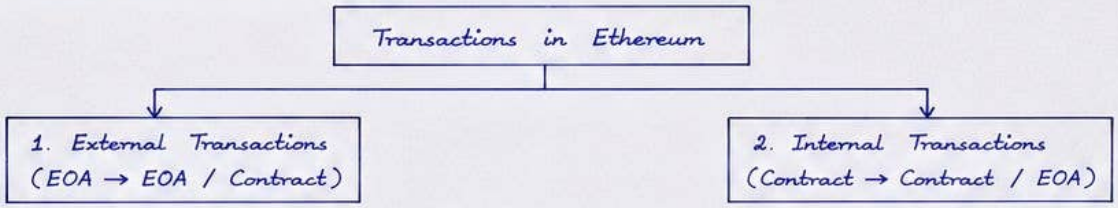
——— X ———

12. Give the different types of Transactions in Ethereum and also explain the fields included in these transactions. 15 Marks

Answer :

In Ethereum, transactions are the messages that are sent to the Ethereum network to trigger some action. Generally, there are two types of transactions.

1. Types of Transactions in Ethereum :



- **External Transactions :** These are initiated by Externally Owned Accounts (EOA). They can transfer Ether, deploy smart contracts or call functions of a contract.
- **Internal Transactions :** These are initiated by smart contracts during the execution of code. They are not recorded in the blockchain explicitly but are part of the execution trace.

2. Fields in Ethereum Transactions :

(A) Fields in External Transactions (Basic Transaction)

Sl.	Field	Description
1.	nonce	Number of transactions sent by the sender account. It prevents replay attacks.
2.	to	20-byte address of the recipient account. In contract creation, this field is empty.
3.	value	Amount of Ether (in wei) to be transferred.
4.	gasLimit	Maximum amount of gas units the sender is willing to provide for the transaction.
5.	gasPrice	Amount of wei to be paid per gas unit.
6.	data	Input data for contract creation or function call (calldata). Empty in simple transfer.
7.	v, r, s	Components of the sender's digital signature. They are used to recover the sender address and verify the transaction.
8.	chainId	ID of the Ethereum network (mainnet, testnet etc.) used for replay protection.

(B) Additional Fields in Contract Creation Transaction

Sl.	Field	Description
1.	init (data)	Initialization code (bytecode) of the contract.
2.	address (to)	Empty (null). The contract address is generated after successful execution.
3.	value	Ether can be sent to the contract at the time of creation.

(C) Internal Transactions :

They do not have standard fields as above. They are Ethereum messages generated during smart contract execution (e.g., call, delegatecall, selfdestruct, create etc.).

3. Key Points :

- External transactions are stored in the blockchain.
- Internal transactions are part of the receipt / execution trace, not stored as transactions.
- All transactions consume gas. The sender pays gas fees.
- The transaction hash (TxHash) uniquely identifies a transaction.

Conclusion :

Ethereum supports two types of transactions: external and internal. External transactions have standard fields like nonce, to, value, gas limit, gas price, data and v, r, s. Internal transactions are generated during smart contract execution.

——— X ———

13. What are Contract creation transaction and Message call transaction? Explain with a suitable diagram.

15 Marks

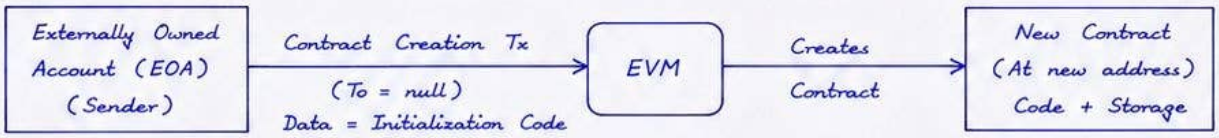
Answer:

In Ethereum, transactions are of two main types based on the action performed.

1. Contract creation transaction
2. Message call transaction

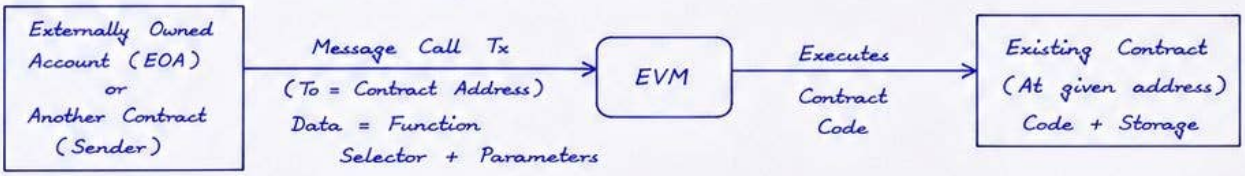
1. Contract Creation Transaction:

- It is a type of transaction used to deploy a new smart contract on the Ethereum blockchain.
- The sender (EOA or another contract) provides the initialization code (bytecode).
- The EVM executes this code, creates the contract and stores it at a new address.
- The transaction field "to" is left empty (null).



2. Message Call Transaction:

- It is used to interact with an existing contract.
- The sender specifies the address of the contract in the "to" field.
- The input data contains the function selector and parameters.
- The EVM executes the contract code and may read / write the storage.
- Ether or data can be sent along with the call.



Comparison:

Aspect	Contract Creation Transaction	Message Call Transaction
Purpose	Deploys a new smart contract	Interacts with an existing contract
To field	Empty (null)	Contains contract address
Data field	Initialization code (bytecode)	Function selector + parameters
Result	New contract address created	Function executed, state may change
Ether Transfer	Optional	Optional
Example	Deploy a token contract	Transfer tokens, call methods

Key Points:

→ Both types of transactions are recorded on the blockchain.
→ Contract creation results in a new address with code and storage.
→ Message call executes existing contract code and may change the state.
→ All transactions consume gas for computation.

Conclusion:

Contract creation transaction is used to deploy a new smart contract, while message call transaction is used to interact with an existing contract by calling its functions.

X

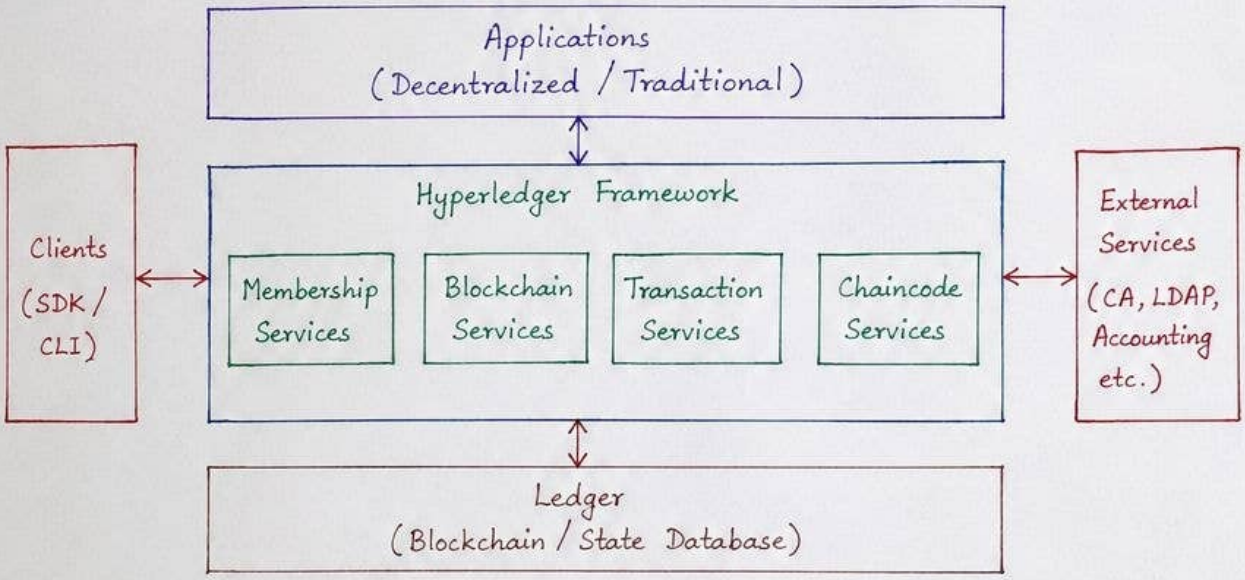
14. Give the architecture of Hyperledger as a protocol and explain.

15 Marks

Answer:

Hyperledger is an open source Blockchain framework hosted by the Linux Foundation. It supports the development of enterprise grade Blockchain applications and frameworks. Hyperledger does not define a Blockchain protocol of its own. Instead, it provides a modular architecture consisting of different components and tools which can be used to build Blockchain solutions.

The architecture of Hyperledger as a protocol is shown below:



Explanation of components:

1) Applications: These are the decentralized or traditional applications developed using Hyperledger frameworks and SDKs.
2) Clients: Applications or users interact with the network through SDK (Software Development Kit) or CLI (Command Line Interface).
3) Hyperledger Framework: It provides the core services required to build Blockchain applications.
   - Membership Services: Manages the identities of participants and their permissions.
   - Blockchain Services: Handles the creation, maintenance and validation of the Blockchain.
   - Transaction Services: Manages the transaction flow, endorsement, ordering and validation.
   - Chaincode Services: Supports the execution of smart contracts (chaincode) on the ledger.
4) Ledger: It is the persistent data store which maintains the Blockchain and the world state.
5) External Services: External systems like Certificate Authority (CA), LDAP, Accounting systems etc. provide additional services to the network.

Thus, Hyperledger architecture provides a modular, flexible and pluggable framework for building enterprise Blockchain solutions.

— X —

15. Discuss about the organization and categories of Hyperledger fabric architecture.

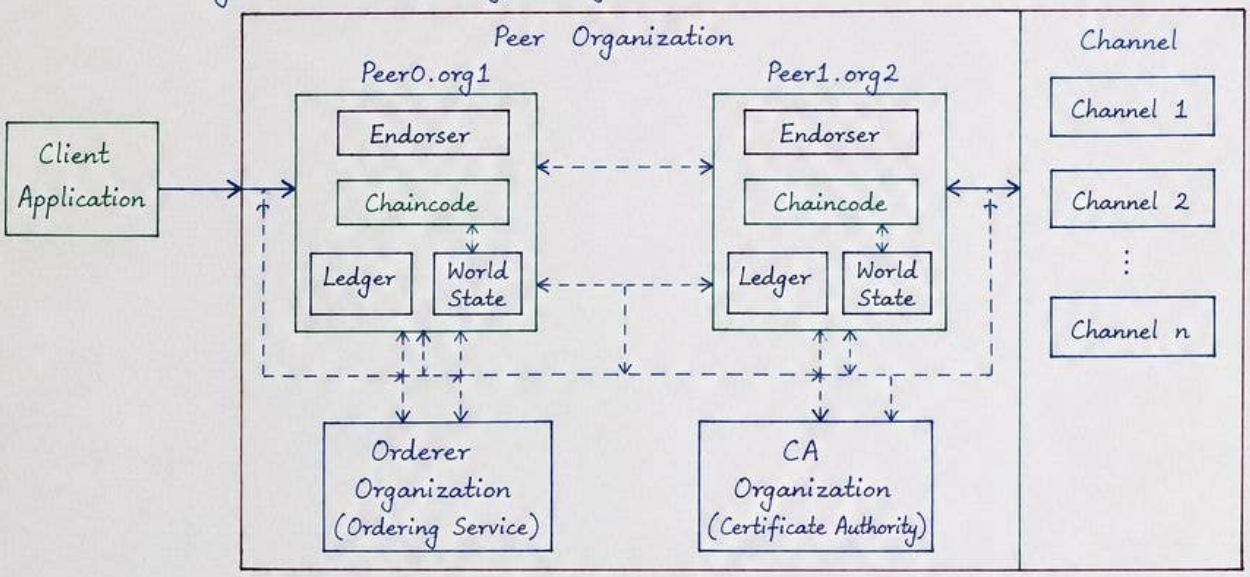
15 Marks

Answer:

Hyperledger Fabric is an enterprise grade permissioned blockchain framework under Linux foundation. Its architecture is modular and designed to support confidentiality, scalability and flexibility. The Fabric architecture can be understood in terms of its organization (how components are arranged and managed) and the categories of the components.

1) Organization of Hyperledger Fabric Architecture :

The organization of Hyperledger Fabric is shown below :



2) Categories (Components) of Hyperledger Fabric Architecture :

The main components are categorized as follows :

i) Client Application : Applications use SDK to submit transaction proposals, evaluate results and submit transactions to the network.

ii) Peer Nodes : Peers are the backbone of the network. Each peer maintains a copy of the ledger and executes chaincode.

- Endorser Peer : Simulates and endorses transaction proposals.
- Committing Peer : Validates blocks and commits to the ledger.
- Ledger : Consists of Blockchain (immutable record) and World State (current state in key-value database).

iii) Ordering Service (Orderer) : Responsible for ordering transaction messages into blocks and distributing them to peers.

iv) Certificate Authority (CA) : Issues digital certificates and manages identities of users, orderers, and admins.

v) Channel : A channel is a private sub-network over which a set of peers communicate. It provides data isolation and confidentiality.

vi) Chaincode : Smart contracts written in languages like Go, Java or Node.js that implement business logic.

vii) MSP (Membership Service Provider) : Manages membership information of identities in the network.

Thus, Hyperledger Fabric architecture is organized into peer organizations, orderer organization and CA organization and consists of components categorized into client, peer, ordering, CA, channel and chaincode.

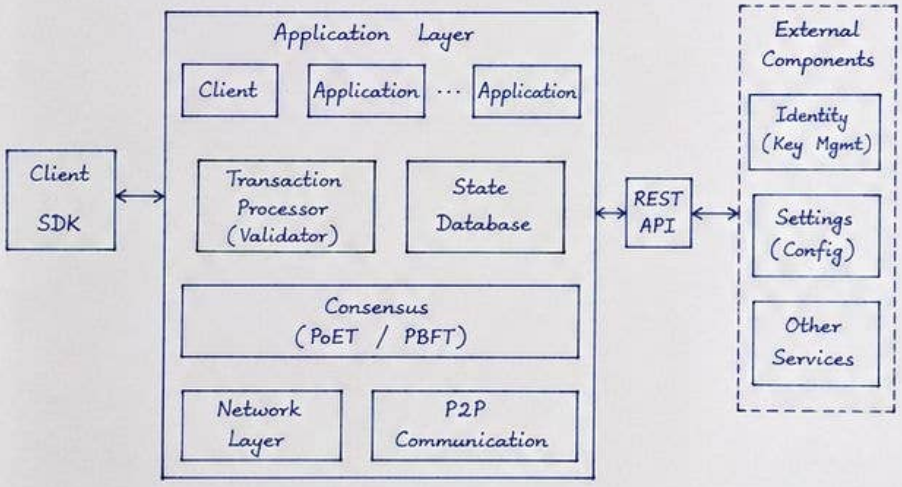
— X —

16. Define Sawtooth lake. Explain the novel concepts of Sawtooth lake. 15 Marks

Answer :

Sawtooth Lake is an open source, modular blockchain platform developed by Intel under the Hyperledger umbrella.
It provides a flexible architecture that enables enterprises to build, deploy and run distributed ledger applications.
It supports permissioned networks and uses a practical Byzantine Fault Tolerance (PBFT) consensus algorithm.

Architecture of Sawtooth Lake :



Novel Concepts of Sawtooth Lake :

1) Transaction Family : Business logic is packaged as a Transaction Family. It allows multiple families to co-exist on the same blockchain.
2) Transactions and Batches : Transactions are grouped into batches and signed by the client. Batches are the unit of work for consensus.
3) Validator (Transaction Processor) : Validates and executes transactions against the state. It is pluggable and written in any language.
4) State Database Merkle Tree : State is stored in a Merkle tree for efficient and cryptographic verification.
5) Consensus - PoET / PBFT : Uses Proof of Elapsed Time (PoET) or Practical Byzantine Fault Tolerance (PBFT) for agreement.
6) REST API : All interactions with the blockchain are done through a REST API.
7) Modularity : Each component is loosely coupled and replaceable.

Thus, Sawtooth Lake introduces modularity, pluggable transaction families, flexible consensus and secure state management as its novel concepts.

— X —

17. What is Corda. List and explain the components of the Corda. 15 Marks

Answer :

Corda is an open source blockchain platform developed by R3 (a consortium of financial institutions). It is designed for business and financial applications and supports permissioned networks. Unlike public blockchains, Corda does not use cryptocurrency. It focuses on privacy, scalability and interoperability.

Components of Corda :

1) Nodes : A node represents a participant in the Corda network. Each node maintains a copy of the ledger and runs applications.
2) Ledger : The ledger is a shared, append-only record of all transactions. It stores the states of all agreements.
3) Contracts : Contracts define the rules and conditions of an agreement. They are written in a JVM language like Java or Kotlin.
4) States : A state represents a piece of data that participants agree on. It is the source of truth in Corda.
5) Transactions : Transactions are proposed changes to the states. They are digitally signed by involved parties.
6) Flow : Flows are the business logic components. They orchestrate transactions between nodes in a secure and reliable manner.
7) Notary Service : The notary ensures that a state is not spent (double spent). It provides uniqueness of states without seeing the state data.
8) Membership Service : It manages identities of participants and issues certificates.
9) CorDa Network Map : It provides information about all the nodes in the network and their legal identities.
10) APIs (Corda RPC / Client API) : It allows client applications to interact with nodes.

Thus, Corda consists of components that provide privacy, legal enforceability, and scalability suitable for enterprise and financial applications.

— X —

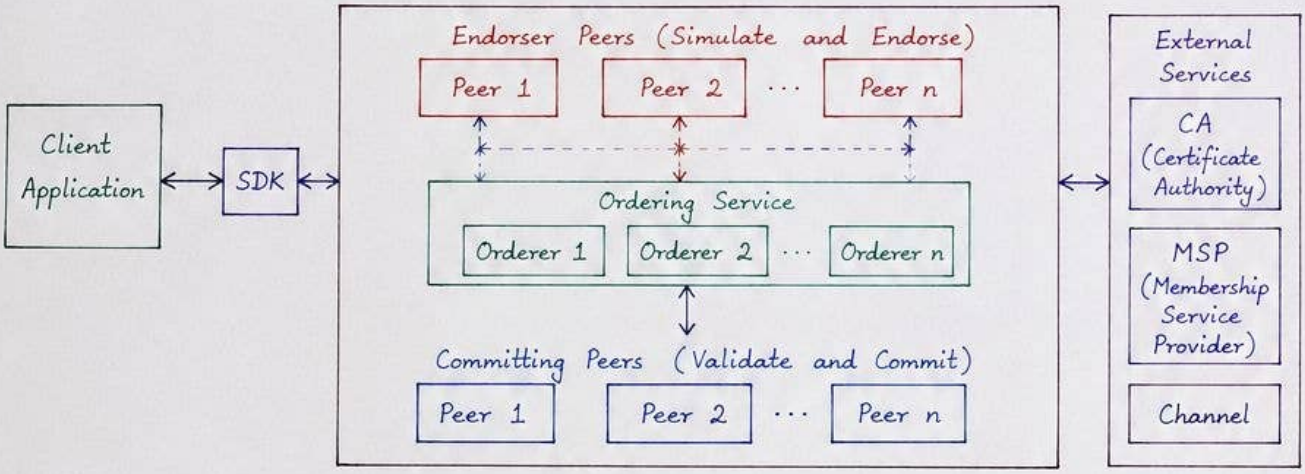
18. Discuss the architecture and key components of Hyperledger fabric.

15 Marks

Answer:

Hyperledger Fabric is an enterprise grade permissioned blockchain framework under Linux foundation. It has a modular architecture and the components are loosely coupled. The architecture supports confidentiality, scalability and flexibility. The major components of Hyperledger Fabric are shown below.

Architecture of Hyperledger Fabric:



Key Components of Hyperledger Fabric:

1) **Client Application (SDK)** : Applications or users interact with the network through SDK (Software Development Kit). It is used to transaction proposals and to queries.
2) **Peers** : Peers are the backbone of the network. They maintain ledger and execute chaincode.
   - **Endorser Peers** : Receive transaction proposals, simulate execution of chaincode and endorse the proposal.
   - **Committing Peers** : Validate endorsed transactions, update the ledger and maintain the world state.
3) **Ordering Service (Orderers)** : They are responsible for ordering transaction messages into blocks. They create a total order of transactions the blocks to peers.
4) **Channel** : A channel is a private sub-network over which a set of peers communicate. It provides data isolation and confidentiality.
5) **Chaincode** : Business logic or smart contracts written in languages like Go, Java or Node.js. It is installed on peers.
6) **Ledger** : Consists of Blockchain (immutable record of transactions) and World State (current state database).
7) **CA (Certificate Authority)** : Issues digital certificates and manages identities of users, peers and orderers.
8) **MSP (Membership Service Provider)** : Manages membership information and validates identities of network an
9) **Policies** : Define rules for endorsing transactions, access control and other operations in the network.

Thus, Hyperledger Fabric architecture is modular with components like clients, peers, orderers, CA, MSP, channels, chaincode and ledger working together to provide a secure, scalable and flexible blockchain platform.

— X —

19. Compare and contrast Hyperledger fabric and Hyperledger sawtooth in terms of architecture consensus and user. 15 Marks

Answer:

Hyperledger Fabric and Hyperledger Sawtooth are two popular blockchain frameworks under the Hyperledger umbrella. Both are open source and support building enterprise grade distributed applications. They differ in their architecture, consensus mechanism and the type of users they are designed for.

Comparison of Hyperledger Fabric and Hyperledger Sawtooth:

Aspect	Hyperledger Fabric	Hyperledger Sawtooth
1. Architecture	• Permissioned blockchain • Modular architecture • Consists of clients, peers (endorsers, committing peers), orderers, channels, chaincode, ledger, CA and MSP • Uses channels for data isolation and privacy • Smart contracts called chaincode (Go, Java, Node.js) • World state stored in a database.	• Permissioned blockchain • Modular and pluggable design • Consists of clients, validators, consensus engine, transaction processor, state database, REST API and SDK • No channels; all transactions go to all validators • Business logic packaged as Transaction Families (Python) • State stored using Merkle tree.
2. Consensus	• Uses ordering service to achieve consensus • Ordering service can implement different protocols like Raft, Kafka, etc. • Uses Practical Byzantine Fault Tolerance (PBFT) or crash fault tolerance depending on the ordering implementation • Transactions are first ordered and then validated and committed.	• Uses Proof of Elapsed Time (PoET) consensus algorithm • PoET is a variant of Practical Byzantine Fault Tolerance (PBFT) • Validators take turns to create blocks based on a wait time • Simple and fast finality • Transactions are validated and committed during block creation.
3. Users	• Designed for enterprises with known identities • Suitable for business networks where participants are known to each other • Offers fine grained access control, privacy and confidentiality • Used in supply chain, banking, trade finance, healthcare, etc.	• Designed for enterprises and developers who want flexibility • Suitable for a wide range of use cases including assets, supply chain, IoT, etc. • Focuses on modularity and ease of integration • Used by organizations building custom blockchain applications quickly.

Similarities:
- Both are open source and part of Hyperledger.
- Both are permissioned blockchains.
- Both support smart contracts and SDKs.
- Both provide REST APIs for integration.

Conclusion: Hyperledger Fabric is more suited for enterprise solutions requiring privacy, channels and complex access control, while Hyperledger Sawtooth is suited for applications requiring high performance, modularity and simplicity in consensus.

— X —

20. What is Hyperledger? Discuss its objectives, structure as a protocol suite and list key projects under the Hyperledger umbrella with a brief description of each. 15 Marks

Answer:

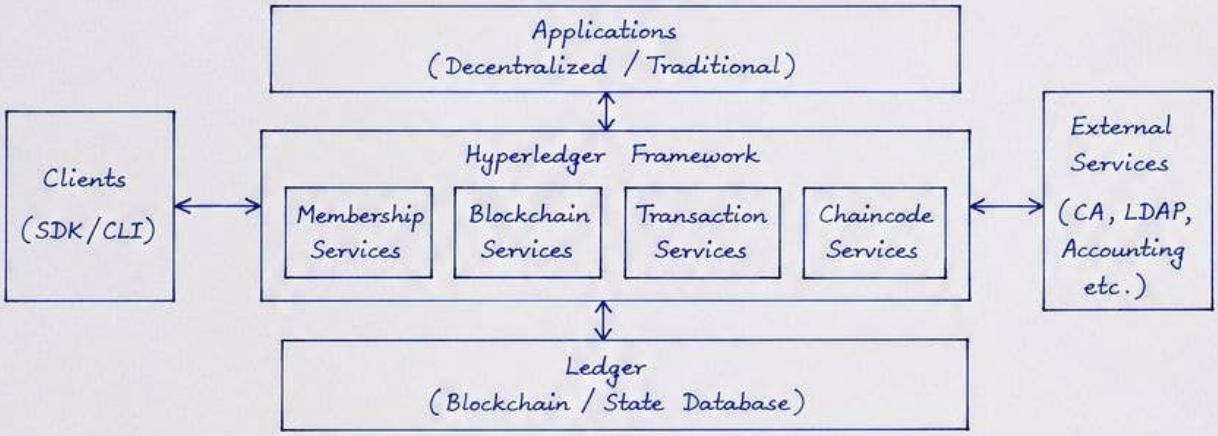
Hyperledger is an open source collaborative effort created to advance cross-industry Blockchain technologies. It is hosted by the Linux Foundation. It provides a modular framework and tools for building enterprise grade Blockchain applications and solutions.

Objectives of Hyperledger:

1) To create an open standard for Blockchain technologies.
2) To provide a platform for developing enterprise grade Blockchain applizations.
3) To encourage collaboration among organizations and industries.
4) To offer a modular architecture with pluggable components.
5) To promote interoperability, confidentiality, scalability and flexibility.

Structure of Hyperledger as a Protocol Suite:

Hyperledger does not define a Blockchain protocol of its own. Instead, it provides a set of modular components and tools that can be used to build Blockchain solutions. The protocol suite is shown below.



Key Projects under Hyperledger umbrella:

1) **Hyperledger Fabric:** A permissioned Blockchain framework for enterprise use. It supports modular architecture, channels, chaincode and smart contracts.
2) **Hyperledger Sawtooth:** A permissioned Blockchain platform with a modular architecture. It uses PoET or PBFT consensus and supports multiple programming languages for smart contracts.
3) **Hyperledger Iroha:** A simple, easy to use Blockchain platform targeting mobile and web applications. It provides commands for creating assets, accounts and querying transactions.
4) **Hyperledger Indy:** A decentralized identity solution. It provides identity management, verifiable credentials and privacy preserving transactions.
5) **Hyperledger Burrow:** An Ethereum based permissioned Blockchain client. It is designed for enterprise use with pluggable consensus.
6) **Hyperledger Composer:** A tool for modeling, deploying and running Blockchain business networks. It simplifies the development of smart contracts.
7) **Hyperledger Caliper:** A benchmarking framework to measure the performance of Blockchain systems. It supports various platforms including Fabric, Sawtooth, Iroha, etc.

Conclusion: Hyperledger provides a robust open source ecosystem of frameworks, tools and libraries for building enterprise Blockchain solutions with emphasis on modularity, security, scalability and interoperability.

— X —

21. Discuss the architecture of Corda. What makes it suitable for financial institutions and how does it handle consensus and privacy.

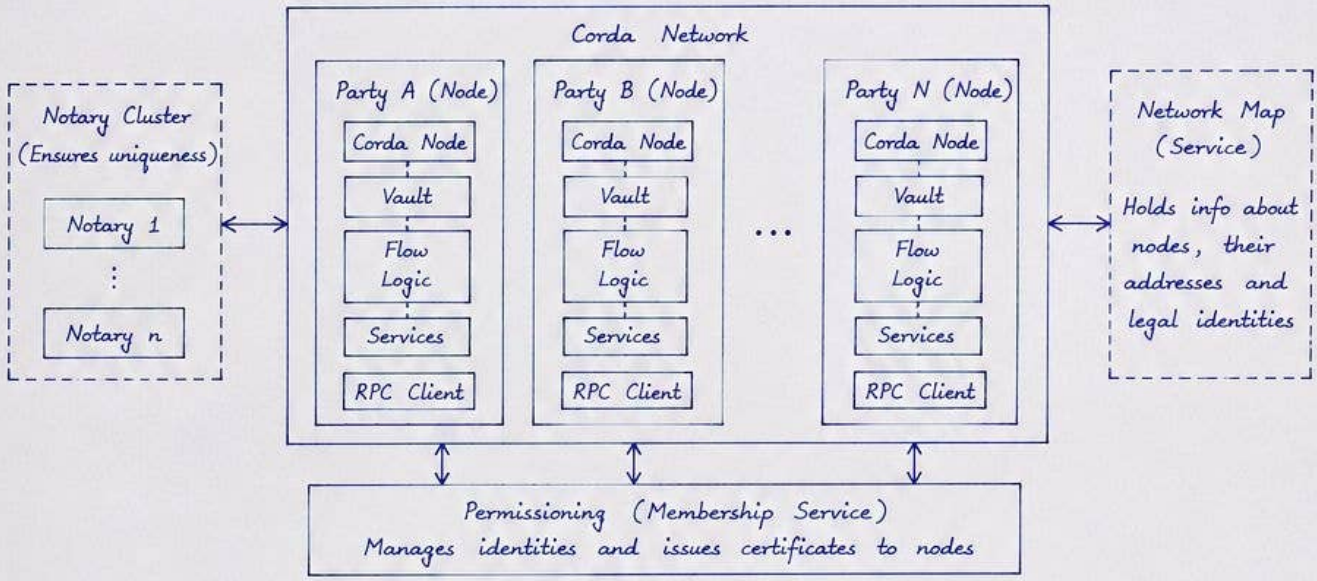
15 Marks

Answer:

Corda is an open source Blockchain platform designed for business and financial applications. It is developed by R3 consortium. Unlike public Blockchain, Corda does not use cryptocurrency. It focuses on privacy, scalability and interoperability for enterprise use cases.

Architecture of Corda:

Corda architecture is shown below.



Why Corda is suitable for financial institutions:

1) **Privacy by design**: Transactions are point-to-point. Only involved parties see the data. Sensitive information is not broadcast to all.
2) **No cryptocurrency**: Corda focuses on business logic and assets, not on tokens, which suits financial use cases.
3) **Scalability**: It supports high throughput as there is no global consensus or mining.
4) **Interoperability**: Corda can interoperate with other systems using APIs and data formats.
5) **Regulatory compliance**: Identity management, auditability and legal enforceability are built in.
6) **Finality of transactions**: Once a transaction is verified and recorded, it is final and cannot be reversed.

How Corda handles consensus:

- Corda does not use global consensus like Proof of Work.
- Instead, it uses a Notary service to provide consensus.
- The notary ensures that each transaction is unique and prevents double spending.
- It checks the transaction and digitally signs it.
- Notary does not see the transaction content, it only validates uniqueness.
- Notary cluster can be used for fault tolerance.

How Corda handles privacy:

- Point-to-point model: Transactions are sent only to the parties involved.
- Only those parties store the transaction in their vaults.
- Others cannot see or access the transaction.
- Data is encrypted during transmission.
- Identity management using Membership Service ensures that only authorized nodes participate.

Conclusion:

Corda architecture with nodes, vaults, flows, notary, network map and membership service provides a secure, private and scalable platform. Its unique consensus through notary and privacy by design makes it highly suitable for financial institutions and enterprise applications.

— X —